

CO 2 – ASSESSMENT TOOL 2 – REPORT

NAME: LAKSHITHA SREE

REGISTER NO : 192511336

COURSE CODE : CSA1717

COURSE NAME : ARTIFICIAL INTELLIGENCE

QUESTION 1: Dry Run of A* Search Algorithm

1. AIM

To perform a dry run of the A* Search algorithm on a given weighted graph and determine the optimal path and total path cost from the Start node (A) to the Goal node (G).

2. OBJECTIVE

- To represent the given problem as a weighted graph with defined nodes, edges, and edge costs.
- To apply heuristic values $h(n)$ for each node and compute the evaluation function $f(n) = g(n) + h(n)$.
- To perform the A* Search algorithm step-by-step, showing the Open List, Closed List, $g(n)$, $h(n)$, and $f(n)$ at every iteration.
- To identify the optimal path from Start to Goal and calculate its total path cost.

3. THEORY

A* Search is an informed (heuristic-based) search algorithm that finds the least-cost path from a start node to a goal node. It maintains two values for every node: $g(n)$, the actual cost accumulated from the start node to node n , and $h(n)$, a heuristic estimate of the cost from node n to the goal. The algorithm always expands the node in the Open List with the lowest evaluation value $f(n) = g(n) + h(n)$. Provided the heuristic is admissible (never overestimates the true cost), A* is guaranteed to return an optimal path.

Problem Formulation:

- Nodes (Variables): A, B, C, D, E, G
- Start Node: A Goal Node: G
- Edges and Costs: A-B = 2, A-C = 4, B-C = 3, B-D = 7, B-E = 2, D-E = 2, D-G = 2, E-G = 2
- Heuristic Values $h(n)$: $h(A)=7$, $h(B)=6$, $h(C)=4$, $h(D)=3$, $h(E)=2$, $h(G)=0$
- Evaluation Function: $f(n) = g(n) + h(n)$

4. ALGORITHM (A* Search Procedure)

Algorithm: A-Star-Search(Graph, Start, Goal)

Input: A weighted graph, a Start node, and a Goal node, along with heuristic values $h(n)$ for every node.

Procedure:

1. Initialize an empty min-priority queue Open List and an empty set Closed List.
2. Set $g(\text{Start}) = 0$ and compute $f(\text{Start}) = g(\text{Start}) + h(\text{Start})$. Insert Start into the Open List.
3. While the Open List is not empty:
 - a. Pop the node N with the lowest $f(N)$ value from the Open List.
 - b. If N is the Goal node, reconstruct and return the optimal path and total cost $g(N)$.
 - c. Move node N to the Closed List.
 - d. For each neighbour M of N not already in the Closed List:
 - i. Compute tentative cost: $\text{tentative_g} = g(N) + \text{cost}(N, M)$.
 - ii. If M is not in the Open List, or tentative_g is lower than the existing $g(M)$, update $g(M) = \text{tentative_g}$, compute $h(M)$, set $f(M) = g(M) + h(M)$, and insert/update M in the Open List.
4. If the Open List becomes empty without reaching the Goal, return failure.

5. INPUT

```
import heapq
```

```
GRAPH = {
```

```
    'A': {'B': 2, 'C': 4},
```

```
    'B': {'A': 2, 'C': 3, 'D': 7, 'E': 2},
```

```
    'C': {'A': 4, 'B': 3, 'E': 3},
```

```
    'D': {'B': 7, 'E': 2, 'G': 2},
```

```
    'E': {'B': 2, 'C': 3, 'D': 2, 'G': 2},
```

```
    'G': {'D': 2, 'E': 2}
```

```
}
```

```
HEURISTIC = {
```

```
    'A': 7,
```

```
    'B': 6,
```

```
    'C': 4,
```

```
    'D': 3,
```

```
    'E': 2,
```

```
    'G': 0
```

```
}
```

```
START = 'A'    # <-- change source node here
```

```
GOAL = 'G'     # <-- change destination node here
```

```

def astar_dry_run(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))

    closed_list = set()
    g_scores = {start: 0}

    iteration = 1
    print("=" * 90)
    print(f'A* SEARCH DRY RUN : START = {start}, GOAL = {goal}')
    print("=" * 90)

    while open_list:
        open_snapshot = sorted(open_list, key=lambda x: x[0])
        open_display = [
            f'{n}(g={g},h={heuristic[n]},f={f})' for f, g, n, p in
open_snapshot
        ]

        f, g, current, path = heapq.heappop(open_list)
        if current in closed_list:
            continue

```

```
print(f"\nIteration {iteration}")
print(f"Current Node: {current}")
print(f"Open List    : {open_display}")
print(f"Closed List   : {sorted(closed_list)}")
print(f"g({current}) = {g}, h({current}) = {heuristic[current]}, "
      f"f({current}) = {f}")
```

```
iteration += 1
```

```
if current == goal:
```

```
    print("\n" + "=" * 90)
    print("GOAL REACHED!")
    print(f"Optimal Path    : {' -> '.join(path)}")
    print(f"Total Path Cost   : {g}")
    print("=" * 90)
    return path, g
```

```
closed_list.add(current)
```

```
for neighbor, cost in graph[current].items():
```

```
    if neighbor in closed_list:
```

```

        continue

    tentative_g = g + cost

    if neighbor not in g_scores or tentative_g < g_scores[neighbor]:

        g_scores[neighbor] = tentative_g

        f_neighbor = tentative_g + heuristic[neighbor]

        heapq.heappush(open_list, (f_neighbor, tentative_g, neighbor,
path + [neighbor]))

    print("No path found.")

    return None, None

astar_dry_run(GRAPH, HEURISTIC, START, GOAL)

```

6. OUTPUT

```

>>> ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/CO3.py =====
A* SEARCH DRY RUN : START = A, GOAL = G
=====

Iteration 1
Current Node   : A
Open List      : ['A(g=0,h=7,f=7)']
Closed List    : []
g(A) = 0, h(A) = 7, f(A) = 7

Iteration 2
Current Node   : B
Open List      : ['B(g=2,h=6,f=8)', 'C(g=4,h=4,f=8)']
Closed List    : ['A']
g(B) = 2, h(B) = 6, f(B) = 8

Iteration 3
Current Node   : E
Open List      : ['E(g=4,h=2,f=6)', 'C(g=4,h=4,f=8)', 'D(g=9,h=3,f=12)']
Closed List    : ['A', 'B']
g(E) = 4, h(E) = 2, f(E) = 6

Iteration 4
Current Node   : G
Open List      : ['G(g=6,h=0,f=6)', 'C(g=4,h=4,f=8)', 'D(g=6,h=3,f=9)', 'D(g=9,h=3,f=12)']
Closed List    : ['A', 'B', 'E']
g(G) = 6, h(G) = 0, f(G) = 6

=====
GOAL REACHED!
Optimal Path   : A -> B -> E -> G
Total Path Cost : 6
=====
>>>

```

7. INTERMEDIATE STEPS (Dry Run)

Iteration 1

- Current Node: A
- Open List: [A]
- Closed List: { }
- $g(A) = 0, h(A) = 7, f(A) = 7$

Expand A $\rightarrow$ neighbours B (cost 2) and C (cost 4):

- $g(B) = 0 + 2 = 2, h(B) = 6, f(B) = 8$
- $g(C) = 0 + 4 = 4, h(C) = 4, f(C) = 8$

Node A is moved to the Closed List.

Iteration 2

- Open List: [B ($f=8$), C ($f=8$)]
- Closed List: {A}

B and C are tied at $f = 8$; B is selected first.

- Current Node: B | $g(B) = 2, h(B) = 6, f(B) = 8$

Expand B $\rightarrow$ neighbours A (visited), C, D, E:

- Via B, $g(C) = 2 + 3 = 5 \rightarrow$ worse than existing $g(C) = 4 \rightarrow$ discarded
- $g(D) = 2 + 7 = 9, h(D) = 3, f(D) = 12$
- $g(E) = 2 + 2 = 4, h(E) = 2, f(E) = 6$

Node B is moved to the Closed List.

Iteration 3

- Open List: [E ($f=6$), C ($f=8$), D ($f=12$)]
- Closed List: {A, B}
- Current Node: E (lowest f -value) | $g(E) = 4, h(E) = 2, f(E) = 6$

Expand E $\rightarrow$ neighbours B (visited), C, D, G:

- Via E, $g(C) = 4 + 3 = 7 \rightarrow$ worse than existing $g(C) = 4 \rightarrow$ discarded
- Via E, $g(D) = 4 + 2 = 6 \rightarrow$ better than existing $g(D) = 9 \rightarrow$ updated: $g(D) = 6, f(D) = 9$

- $g(G) = 4 + 2 = 6$, $h(G) = 0$, $f(G) = 6$

Node E is moved to the Closed List.

Iteration 4

- Open List: [G ($f=6$), C ($f=8$), D ($f=9$)]
- Closed List: {A, B, E}
- Current Node: G (lowest f -value) | $g(G) = 6$, $h(G) = 0$, $f(G) = 6$

G is the Goal node — the search terminates here.

Summary Table of All Iterations:

Iteration	Current Node	$g(n)$	$h(n)$	$f(n)$	Open List	Closed List
1	A	0	7	7	[A]	{ }
2	B	2	6	8	[B, C]	{A}
3	E	4	2	6	[E, C, D]	{A, B}
4	G	6	0	6	[G, C, D]	{A, B, E}

8. RESULT

Optimal Path and Cost:

- Optimal Path: $A \rightarrow B \rightarrow E \rightarrow G$
- Cost Breakdown: $A \rightarrow B$ (2) + $B \rightarrow E$ (2) + $E \rightarrow G$ (2)
- Total Path Cost: 6 units

This matches $f(G) = 6$ exactly, since $h(G) = 0$, confirming that the path found is optimal — a guaranteed property of A* Search when the heuristic used is admissible.

QUESTION 2: Dry Run of Minimax Algorithm with Alpha-Beta Pruning

1. AIM

To perform a dry run of the Minimax algorithm with Alpha-Beta Pruning on a given game tree, evaluated left to right, and to determine the best move for MAX along with the final minimax value.

2. OBJECTIVE

- To represent the given two-ply game tree with a MAX root and two MIN child nodes, each having three terminal (leaf) values.
- To apply the Minimax procedure combined with Alpha-Beta Pruning, tracking α (alpha) and β (beta) at every node.
- To identify and justify which branches of the tree get pruned during evaluation.
- To determine the best move for MAX and state the final minimax value of the game tree.

3. THEORY

Minimax is a decision-making algorithm used in two-player, zero-sum games. It assumes MAX tries to maximize the outcome while MIN tries to minimize it, and the value of each node is propagated up the tree accordingly. Alpha-Beta Pruning is an optimization of Minimax that eliminates branches which cannot influence the final decision, without changing the result. It maintains two bounds: α (alpha), the best value MAX can currently guarantee along the path, and β (beta), the best value MIN can currently guarantee along the path. Whenever $\beta \leq \alpha$ at any node, the remaining children of that node are pruned, since the opposing player would never allow that branch to be reached.

Problem Formulation:

- Root Node: MAX, with two children MIN1 and MIN2
- MIN1 (children / leaf values): 3, 5, 6
- MIN2 (children / leaf values): 9, 1, 2
- Evaluation Order: Left to right

4. ALGORITHM (Minimax with Alpha-Beta Pruning)

Algorithm: Alpha-Beta(node, α , β , maximizingPlayer)

Input: A game tree with a MAX root, alternating MIN/MAX levels, and terminal (leaf) values.

Output: The minimax value of the root, the best move for MAX, and the list of pruned branches.

Procedure:

- 1. If the node is a terminal (leaf) node, return its value.
- 2. If the node is a MAX node:
 - a. Set $\text{value} = -\infty$.
 - b. For each child, recursively evaluate Alpha-Beta(child, α , β , MIN) and take $\text{value} = \max(\text{value}, \text{childValue})$.
 - c. Update $\alpha = \max(\alpha, \text{value})$.
 - d. If $\beta \leq \alpha$, stop evaluating remaining children (prune) and return value.
- 3. If the node is a MIN node:
 - a. Set $\text{value} = +\infty$.
 - b. For each child, recursively evaluate Alpha-Beta(child, α , β , MAX) and take $\text{value} = \min(\text{value}, \text{childValue})$.
 - c. Update $\beta = \min(\beta, \text{value})$.
 - d. If $\beta \leq \alpha$, stop evaluating remaining children (prune) and return value.
- 4. Return the propagated value up to the root as the final minimax value.

5. INPUT

```
TREE = [  
    [3, 5, 6], # children of first MIN node  
    [9, 1, 2] # children of second MIN node  
]
```

```
pruned_nodes = []
```

```
def alphabeta(node, depth, alpha, beta, maximizing, path_label="ROOT"):
```

```
    indent = " " * depth
```

```
    if isinstance(node, int):
```

```

print(f'{indent}Leaf {path_label} = {node}')
return node

if maximizing:
    value = float('-inf')
    print(f'{indent}Entering MAX node [{path_label}] (alpha={alpha},
beta={beta})')
    for i, child in enumerate(node):
        child_label = f'{path_label}-{i + 1}'
        child_val = alphabeta(child, depth + 1, alpha, beta, False, child_label)
        value = max(value, child_val)
        alpha = max(alpha, value)
        print(f'{indent} -> After child {child_label}: value={value}, "
            f'alpha={alpha}, beta={beta}')
        if beta <= alpha:
            remaining = node[i + 1:]
            if remaining:
                print(f'{indent} ** PRUNING remaining children of {path_label}: "
                    f'{remaining} (beta <= alpha)')
                pruned_nodes.append((path_label, remaining))
            break
    print(f'{indent}MAX node [{path_label}] selected value = {value}')
    return value

else:
    value = float('inf')
    print(f'{indent}Entering MIN node [{path_label}] (alpha={alpha},
beta={beta})')
    for i, child in enumerate(node):
        child_label = f'{path_label}-{i + 1}'
        child_val = alphabeta(child, depth + 1, alpha, beta, True, child_label)
        value = min(value, child_val)

```

```

    beta = min(beta, value)
    print(f'{indent} -> After child {child_label}: value={value}, "
          f"alpha={alpha}, beta={beta}")
    if beta <= alpha:
        remaining = node[i + 1:]
        if remaining:
            print(f'{indent} ** PRUNING remaining children of {path_label}: "
                  f"{remaining} (beta <= alpha)")
            pruned_nodes.append((path_label, remaining))
        break
    print(f'{indent}MIN node [{path_label}] selected value = {value}')
    return value

```

```

print("=" * 90)
print("MINIMAX WITH ALPHA-BETA PRUNING DRY RUN")
print("=" * 90)

```

```

best_value = alphabeta(TREE, 0, float('-inf'), float('inf'), True, "MAX")

```

```

print("\n" + "=" * 90)
print(f'FINAL MINIMAX VALUE : {best_value}')
print(f'PRUNED NODES      : {pruned_nodes if pruned_nodes else 'None'}')
print("=" * 90)

```

6. OUTPUT

```
>>> ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/CO3.py =====
=====
MINIMAX WITH ALPHA-BETA PRUNING DRY RUN
=====
Entering MAX node [MAX] (alpha=-inf, beta=inf)
  Entering MIN node [MAX-1] (alpha=-inf, beta=inf)
    Leaf MAX-1-1 = 3
    -> After child MAX-1-1: value=3, alpha=-inf, beta=3
    Leaf MAX-1-2 = 5
    -> After child MAX-1-2: value=3, alpha=-inf, beta=3
    Leaf MAX-1-3 = 6
    -> After child MAX-1-3: value=3, alpha=-inf, beta=3
  MIN node [MAX-1] selected value = 3
  -> After child MAX-1: value=3, alpha=3, beta=inf
  Entering MIN node [MAX-2] (alpha=3, beta=inf)
    Leaf MAX-2-1 = 9
    -> After child MAX-2-1: value=9, alpha=3, beta=9
    Leaf MAX-2-2 = 1
    -> After child MAX-2-2: value=1, alpha=3, beta=1
    ** PRUNING remaining children of MAX-2: [2] (beta <= alpha)
  MIN node [MAX-2] selected value = 1
  -> After child MAX-2: value=3, alpha=3, beta=inf
MAX node [MAX] selected value = 3

=====
FINAL MINIMAX VALUE : 3
PRUNED NODES       : [('MAX-2', [2])]
=====
>>>
```

7. INTERMEDIATE STEPS (Dry Run)

Branch 1: MIN1 (children 3, 5, 6)

MIN1 begins with $\alpha = -\infty$, $\beta = +\infty$ (inherited from the root).

- Leaf 3: value = 3 $\rightarrow$ MIN1's value = $\min(+\infty, 3) = 3 \rightarrow \beta = 3$. Check: $\beta(3) \leq \alpha(-\infty)$? No $\rightarrow$ continue.
- Leaf 5: value = 5 $\rightarrow$ MIN1's value = $\min(3, 5) = 3$ (no change). β remains 3.
- Leaf 6: value = 6 $\rightarrow$ MIN1's value = $\min(3, 6) = 3$ (no change). β remains 3.

MIN1 finishes all three children $\rightarrow$ MIN1 = 3

Back at the root (MAX): $\alpha = \max(-\infty, 3) = 3$

Branch 2: MIN2 (children 9, 1, 2)

MIN2 begins with $\alpha = 3$ (inherited from MAX after Branch 1), $\beta = +\infty$.

- Leaf 9: value = 9 $\rightarrow$ MIN2's value = $\min(+\infty, 9) = 9 \rightarrow \beta = 9$. Check: $\beta(9) \leq \alpha(3)$? No $\rightarrow$ continue.
- Leaf 1: value = 1 $\rightarrow$ MIN2's value = $\min(9, 1) = 1 \rightarrow \beta = 1$. Check: $\beta(1) \leq \alpha(3)$? YES $\rightarrow$ PRUNE remaining child.

Since β (1) is now less than or equal to α (3), MIN2 can never produce a result better than 1 for MAX, while MAX is already guaranteed at least 3 from Branch 1. Therefore the third child of MIN2 (leaf value 2) is never evaluated — it is pruned.

MIN2 = 1 (final value; the leaf '2' is skipped)

Back at the root (MAX): $\alpha = \max(3, 1) = 3$ (no change — 3 remains the best option for MAX)

Alpha-Beta Table:

Node	α (start)	β (start)	Value Computed	α (end)	β (end)	Pruned?
MIN1	$-\infty$	$+\infty$	$\min(3,5,6) = 3$	$-\infty$	3	No
MAX (after MIN1)	$-\infty$	$+\infty$	-	3	$+\infty$	-
MIN2	3	$+\infty$	$9 \rightarrow 1$ (then pruned)	3	1	Yes
MAX (final)	3	$+\infty$	$\max(3,1) = 3$	3	$+\infty$	-

Why the Pruning Happened:

At MIN2, after evaluating the leaf value 1, MIN2's value can only stay at 1 or decrease further, since MIN always chooses the minimum. MAX has already secured a guaranteed value of 3 from Branch 1 (MIN1). Since MIN2 can never exceed 1 at this stage, MAX would never choose this branch over the already-known value of 3, so exploring the remaining leaf (value 2) under MIN2 is unnecessary and is pruned. This is a classic beta cutoff, triggered inside a MIN node when $\beta \leq \alpha$.

8. RESULT

Result	Value
Best Move for MAX	Branch 1 (MIN1)
Final Minimax Value	3
Pruned Node(s)	Leaf value 2 (third child of MIN2)

Conclusion:

- MAX should choose Branch 1 (the MIN1 subtree), since it guarantees a minimax value of 3, the highest value MAX can guarantee regardless of MIN's choices.
- Alpha-Beta Pruning allowed one leaf node (value 2) to be skipped without affecting the correctness of the final result.